



← Back to student guides

GitHub Actions

CI/CD

3 levels

139 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

EVENT

WORKFLOW

JOB

STEP

21

sections

30

examples





is repeated for its own sake. where you already know the basics, we go straight to the depth.

Where this picks up

Topic you already use	What this level adds
Triggers and filters	Activity types, <code>workflow_run</code> , <code>repository_dispatch</code> , the required-check trap
<code>needs</code>	The full dependency graph, <code>needs.*.result</code> , fan-in and fan-out
<code>run</code> / <code>uses</code>	Shell selection, exit codes, <code>continue-on-error</code> , <code>outcome</code> VS <code>conclusion</code>
Expressions and <code>if</code>	Every context, operator, and function; precedence rules
Environment variables	Scope precedence, default variables, all four special files
Secrets	Secrets vs variables, three definition levels, masking, fork behaviour
Caching	Key design and its two failure modes, scope, <code>cache-hit</code> , per-language recipes
Artifacts	Retention, name collisions, empty uploads, cross-run download
Job outputs	Size limits, passing JSON, driving a matrix
Matrix	<code>include</code> , <code>exclude</code> , <code>fail-fast</code> , <code>max-parallel</code> , dynamic generation
Guards	Concurrency control and cancellation strategy
Debugging	Context dumps, debug logging, cache bypass, bisecting
new	Service containers · container jobs · reusable workflows · composite actions · environments and approvals



You have used `${ }` for a handful of values. The full set of contexts:

Context	Available in	Holds
github	Everywhere	<code>ref</code> , <code>ref_name</code> , <code>ref_type</code> , <code>sha</code> , <code>actor</code> , <code>event_name</code> , <code>repository</code> , <code>repository_owner</code> , <code>run_id</code> , <code>run_number</code> , <code>run_attempt</code> , <code>workspace</code> , <code>job</code> , <code>event.*</code>
env	Steps, <code>if</code>	Variables you defined
vars	Everywhere	Organisation, repository, and environment variables
secrets	Everywhere except step <code>if</code>	Encrypted values
job	Steps	<code>job.status</code> , <code>job.services.*</code>
jobs	Reusable workflow <code>outputs</code>	Callee job results
steps	Later steps in the job	<code>steps.<id>.outputs.*</code> , <code>.outcome</code> , <code>.conclusion</code>
runner	Steps	<code>os</code> , <code>arch</code> , <code>name</code> , <code>temp</code> , <code>tool_cache</code>
needs	Dependent jobs	<code>needs.<job>.outputs.*</code> , <code>needs.<job>.result</code>
strategy	Matrix jobs	<code>job-index</code> , <code>job-total</code> , <code>fail-fast</code>
matrix	Matrix jobs	The current combination
inputs	<code>workflow_dispatch</code> , <code>workflow_call</code>	Declared inputs

Operators and functions



```
# Comparison and logic
if: github.event_name == 'push' && github.ref != 'refs/heads/main'
if: contains(fromJSON('["main","develop"]'), github.ref_name)
if: github.actor != 'dependabot[bot]' || inputs.force == true

# String functions
if: startsWith(github.ref, 'refs/tags/v')
if: endsWith(github.repository, '-internal')
if: contains(github.event.head_commit.message, '[skip ci]')

# format() builds strings; join() flattens arrays
- run: echo "${{ format('{0}-{1}', runner.os, matrix.python) }}"
- run: echo "${{ join(matrix.python, ', ') }}"

# hashFiles() is the backbone of a cache key
key: "${{ runner.os }}-${{ hashFiles('**/requirements.txt', '**/pyproject.toml') }}"

# toJSON / fromJSON convert between strings and structures
- run: echo "${{ toJSON(github.event) }}"
matrix:
  target: "${{ fromJSON(needs.discover.outputs.targets) }}"
```

Two precedence rules worth memorising

Inside `if:` the `${{ }}` wrapper is **optional** because the value is already evaluated as an expression; everywhere else it is required, and mixing the two styles in one condition produces confusing partial evaluation. `&&` / `||` return **operands, not booleans**, which is why `${{ inputs.tag || github.sha }}` works as a default-value idiom.

► TRY IT

1. Print three contexts side by side in one step: `toJSON(github.event_name)` , `toJSON(runner)` , and `toJSON(needs)` .
2. Use the default-value idiom: add a `workflow_dispatch` input `tag` and echo `${{ inputs.tag || github.sha }}` , once with the input filled and once empty.
3. Try `contains(fromJSON('["main","develop"]'), github.ref_name)` as an `if` on a step, from two different branches.



empty object in a job with no dependencies, which is worth knowing before you debug an empty value.

Step results: **outcome** versus **conclusion**

Beginner used `if: failure()`. The precise mechanics matter once you tolerate failures deliberately.

YAML

Copy

```
- id: flaky
  continue-on-error: true
  run: ./might-fail.sh

- name: React to the REAL result
  if: steps.flaky.outcome == 'failure'
  run: echo "it failed, but we chose to continue"
```

Field	Value after a tolerated failure
<code>steps.flaky.outcome</code>	<code>failure</code> : what the step did
<code>steps.flaky.conclusion</code>	<code>success</code> : the result after <code>continue-on-error</code> is applied

The same distinction exists at job level as `needs.<job>.result`, which is `success`, `failure`, `cancelled`, or `skipped`.

Status functions in full

Function	True when
<code>success()</code>	Every previous step/job succeeded: the implicit default



<code>failure()</code>	Any previous step/job failed
<code>cancelled()</code>	The run was cancelled
<code>always()</code>	Always, including cancellation

`always()` is stronger than people expect

It runs even when the workflow is **cancelled**, which means an `always()` step can keep a run alive that someone is trying to stop. For cleanup that should respect cancellation, prefer `if: !cancelled()` .

► TRY IT

1. Add a step with `id: flaky` , `continue-on-error: true` , and `run: exit 1` .
2. Follow it with a step printing both values: `echo "outcome=${{ steps.flaky.outcome }} conclusion=${{ steps.flaky.conclusion }}"` .
3. Now add a step with `if: always()` and cancel the run from the Actions tab while it is going.

Expected result — `outcome=failure conclusion=success` , and the job is green. The cancelled run still executes the `always()` step, which is why `!cancelled()` exists for cleanup that should stop.

The four special files

Beginner used `$GITHUB_ENV` and `$GITHUB_STEP_SUMMARY` . There are four, and each solves a different problem.

YAML

Copy

```
- id: meta
  run: |
```



```
# 2. step outputs, addressed by the step's id
echo "artifact=dist/app.tar.gz" >> "$GITHUB_OUTPUT"

# 3. prepend to PATH for later steps
echo "$HOME/.local/bin" >> "$GITHUB_PATH"

# 4. Markdown rendered on the run page
echo "### Build complete" >> "$GITHUB_STEP_SUMMARY"

- run: echo "$IMAGE_TAG" # from GITHUB_ENV
- run: echo "${{ steps.meta.outputs.artifact }}" # from GITHUB_OUTPUT
- run: my-tool --version # found via GITHUB_PATH
```

Multi-line values need a delimiter, or the file format breaks:

YAML

Copy

```
- run: |
  {
    echo 'NOTES<<EOF'
    cat CHANGELOG.md
    echo 'EOF'
  } >> "$GITHUB_ENV"
```

Two limits

A value written to `$GITHUB_ENV` is **not** readable in the step that wrote it, only from the next step onwards. None of these four cross a **job** boundary; that needs job outputs or artifacts.

► TRY IT

1. Write one step that appends to all four files, then later steps that read each back.
2. Include a multi-line value using the `EOF` delimiter form, then print it.
3. Add a second job with `needs` and try to read the `$GITHUB_ENV` value there.

Expected result – the first three work from the next step onwards, the summary renders on the run page, and the second job sees nothing. That last result is the



Environment variables: precedence and defaults

Three scopes, most specific wins, and there is a fourth source you did not declare.

YAML

Copy

```
env:
  LOG_LEVEL: info          # 3rd priority: workflow
jobs:
  test:
    env:
      LOG_LEVEL: debug      # 2nd priority: job
    steps:
      - run: ./run.sh
        env:
          LOG_LEVEL: trace # 1st priority: step – wins
```

GitHub also injects a set of default variables into every step:

Variable	Contains
GITHUB_REPOSITORY	owner/repo
GITHUB_REF / GITHUB_REF_NAME	refs/heads/main / main
GITHUB_SHA	The triggering commit
GITHUB_WORKSPACE	Checkout directory
GITHUB_RUN_ID / GITHUB_RUN_NUMBER / GITHUB_RUN_ATTEMPT	Run identity
GITHUB_EVENT_PATH	Path to the full event payload JSON
RUNNER_OS / RUNNER_TEMP / RUNNER_ARCH	Machine facts



```
# Shell-safe short SHA, without an expression
- run: echo "tag=sha-${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"

# The whole event payload is on disk
- run: jq '.pull_request.labels[].name' "$GITHUB_EVENT_PATH"
```

► TRY IT

1. Set the same variable at workflow, job, and step level with three different values, then echo it from the step.
2. Print the injected defaults: `echo "$GITHUB_REF_NAME $GITHUB_RUN_ATTEMPT $RUNNER_OS" .`
3. Dump the raw event: `jq '.repository.full_name, .ref' "$GITHUB_EVENT_PATH" .`
4. Re-run the same job and watch `GITHUB_RUN_ATTEMPT` change.

Expected result – the step-level value wins, and you have found the event payload on disk, often faster to explore with `jq` than through expressions.

Secrets and variables

Beginner treated secrets as one thing. There are two kinds, and conflating them is a real security problem.

✓ **Secrets:** `${{ secrets.NAME }}`

- Encrypted at rest, write-only in the interface
- Masked in logs on a best-effort basis
- **Not** given to fork pull requests
- For tokens, keys, passwords, connection strings

✗ **Variables:** `${{ vars.NAME }}`

- Plain text, readable by anyone with repo access
- Printed normally in logs
- Available in every run, forks included
- For regions, image names, URLs, feature flags

Both exist at **three levels**, most specific winning, which is more useful than it first appears:



AWS_REGION=eu-west-1 AWS_REGION=eu-west-2 AWS_REGION=us-east-1
(default for 60 repos) (this repo overrides) (production overrides again)

YAML

Copy

```
jobs:
  deploy:
    environment: production          # unlocks environment-scoped values
    runs-on: ubuntu-latest
    steps:
      - env:
          TOKEN: ${ secrets.DEPLOY_TOKEN } # environment secret
        run: ./deploy.sh --region "${ vars.AWS_REGION }"
```

Masking is a safety net, not a mechanism

GitHub redacts **known** secret values from logs. It cannot redact a value you transformed: base64-encode a secret and print it and the redaction fails. Never print secrets, and pass them via `env` so they never appear in a command line.

► TRY IT

1. Create a repository **variable** `REGION=eu-west-1` and a repository **secret** `TOKEN=abc123`.
2. Echo both. Note which one is redacted.
3. Create an **environment** called `staging` with its own `REGION=us-east-1`, add `environment: staging` to the job, and echo it again.
4. Try `echo "$TOKEN" | rev` and look at the log.

Expected result – the variable prints, the secret shows `***`, the environment value overrides the repository one, and the reversed secret is printed in full. That last line is the proof that masking is a safety net rather than a mechanism.



Beginner used `cache: pip` . That one line hides a mechanism you can build yourself.

YAML

Copy

```
- uses: actions/cache@v4
  id: deps
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}
    restore-keys: |
      ${{ runner.os }}-pip-
```

Piece	Behaviour
<code>key</code>	Exact match. A hit restores and skips the save at the end of the job
<code>restore-keys</code>	Ordered prefixes tried on a miss: a partial hit, still useful
<code>path</code>	What gets archived. The dependency <code>cache</code> , not the installed tree
Immutability	An entry is never overwritten, so the key must change when content should
Scope	Follows the branch graph: a branch reads its own and its base's caches
<code>cache-hit</code>	<code>'true'</code> only on an exact key match

There are exactly two ways to get this wrong, and they fail in opposite directions:

✓ Keys that work

- `${{ runner.os }}-pip-${{ hashFiles('**/requirements*.txt') }}`
- Add the tool version when it changes the layout: `-py${{ matrix.python }}`
- Always pair with a `restore-keys` prefix

✗ Keys that waste money

- `github.sha` or `run_id` : misses **every** run
- A fixed string: never invalidates, ships **stale** dependencies
- `hashFiles('**')` : any source edit busts the dependency cache



YAML

Copy

```
- uses: actions/cache@v4
  id: models
  with:
    path: ~/.cache/huggingface
    key: hf-${{ hashFiles('models.lock') }}
    restore-keys: hf-

- name: Download only on an exact miss
  if: steps.models.outputs.cache-hit != 'true'
  run: python scripts/fetch_models.py
```

Per-language recipes

Language	Path to cache	Key on
Python (pip)	<code>~/.cache/pip</code>	<code>requirements*.txt</code> , <code>poetry.lock</code>
Node	<code>~/.npm</code>	<code>package-lock.json</code>
Java (Maven)	<code>~/.m2/repository</code>	<code>pom.xml</code>
Go	<code>~/.cache/go-build</code> , <code>~/go/pkg/mod</code>	<code>go.sum</code>
Rust	<code>~/.cargo</code> , <code>target/</code>	<code>Cargo.lock</code>

Do not cache an installed tree

Cache the **download** directory, not `node_modules` or a virtualenv. Restoring an installed tree brings platform-specific binaries and half-resolved state with it, and produces failures that make no sense against the diff.

► TRY IT

1. Add `actions/cache` with a good key and print `steps.deps.outputs.cache-hit` . Run twice.
2. Now change the key to include `${{ github.sha }}` and run twice more.



Expected result – the good key misses then hits. The SHA key misses both times. The fixed key hits even though the lockfile changed, shipping stale dependencies with no warning. Seeing all three once makes key design obvious forever.

Artifacts: retention, collisions, and emptiness

YAML

Copy

```
- uses: actions/upload-artifact@v4
  if: always()
  with:
    name: coverage-${{ matrix.python }} # unique per matrix cell
    path: |
      htmlcov/
      reports/junit.xml
    retention-days: 7 # override the 90-day default
    if-no-files-found: error # fail loudly, not silently empty
    compression-level: 9
```

Habit	Reason
<code>if: always()</code>	A green run's report is nice; a red run's report is the point
Unique name per matrix cell	In v4, two uploads with the same name conflict rather than merging
Short <code>retention-days</code> for CI noise	Artifacts count against repository storage
<code>if-no-files-found: error</code>	Turns a silently empty artifact into a visible failure

Downloading is the mirror image, and can reach across runs:



```
# Everything from this run
- uses: actions/download-artifact@v4
  with:
    path: ./all-artifacts
    merge-multiple: true

# One artifact from a DIFFERENT run
- uses: actions/download-artifact@v4
  with:
    name: dist
    run-id: ${ github.event.workflow_run.id }
    github-token: ${ secrets.GITHUB_TOKEN }
```

► TRY IT

1. Run a two-cell matrix where both cells upload an artifact with the **same** name.
2. Fix it by suffixing with `${ matrix.python }` .
3. Point `path:` at a directory that does not exist, first without and then with `if-no-files-found: error` .

Expected result – the duplicate name produces a conflict error in v4 rather than merging. The missing path silently succeeds until you add `if-no-files-found: error` , which is why it belongs on every upload.

The job dependency graph

`needs` does more than order jobs. It builds a graph you can fan out from and fan back into.

YAML

Copy

```
jobs:
  setup:
    runs-on: ubuntu-latest
    outputs:
```



```
- id: v
  run: |
    echo "version=1.4.${{ github.run_number }}" >> "$GITHUB_OUTPUT"
    echo 'targets=["api","worker","web"]' >> "$GITHUB_OUTPUT"

build:                                # fan out: three parallel jobs
  needs: setup
  strategy:
    matrix:
      target: ${{ fromJSON(needs.setup.outputs.targets) }}
  runs-on: ubuntu-latest
  steps:
    - run: make build TARGET=${{ matrix.target }} VERSION=${{ needs.setup.outputs.vers

report:                                # fan in: runs whatever happened
  needs: [setup, build]
  if: always()
  runs-on: ubuntu-latest
  steps:
    - run: |
        echo "setup : ${{ needs.setup.result }}"
        echo "build : ${{ needs.build.result }}"
    - if: needs.build.result == 'failure'
      run: ./notify.sh
```

► TRY IT

1. Build the fan-out/fan-in example: a `setup` job emitting JSON, a matrix `build` consuming it, and a `report` job with `if: always()` .
2. Print `needs.build.result` in the report job.
3. Make one matrix cell fail, then re-run and inspect the reported result.



that aggregation surprises people the first time they rely on it.

Matrix in depth

YAML

Copy

```
jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      fail-fast: false
      max-parallel: 4
      matrix:
        os: [ubuntu-latest, macos-latest]
        python: ['3.11', '3.12']
        include:
          - os: ubuntu-latest      # add one non-cartesian combination
            python: '3.13'
            experimental: true
          - os: ubuntu-latest      # or attach extra values to an existing one
            python: '3.12'
            coverage: true
        exclude:
          - os: macos-latest
            python: '3.11'
      continue-on-error: ${ matrix.experimental = true }
    name: test (${ matrix.os }, py${ matrix.python })
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python }
      - run: pytest -q
      - if: matrix.coverage == true
        run: pytest --cov --cov-report=xml
```




Axes	Multiply. 2 OSes x 2 Python3 = 4 jobs
<code>include</code>	Adds a combination, or attaches extra keys to an existing one
<code>exclude</code>	Removes a generated combination
<code>fail-fast</code>	<code>true</code> by default: one failure cancels every sibling
<code>max-parallel</code>	Caps concurrent cells: use when they share a database or API quota
Ceiling	256 jobs per workflow run

Adding an axis renames a required check

The generated job name embeds the combination, so branch protection that requires `test (3.11)` starts waiting forever when the name becomes `test (ubuntu-latest, 3.11)`. Set an explicit `name:` on any matrix job used as a required check.

Dynamic matrices

The monorepo pattern: discover what changed, then build only that.



► TRY IT

1. Build a 2×2 matrix, then add one `include` and one `exclude`. Count the resulting jobs before you run it, then check.
2. Attach `experimental: true` via `include` and wire up `continue-on-error`.
3. Add a third axis temporarily and look at how many jobs appear.
4. Set an explicit `name:` and note how the Actions tab changes.

Expected result – four jobs, minus one, plus one. The third axis produces a wall of jobs that makes the "keep it to two axes" advice self-evident, and the explicit name is what keeps branch protection working.

Service containers

Mocking a database in integration tests is a compromise. `services` starts real containers on the job's network before your steps run.

YAML

Copy

```
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: app_test
```



```
--health-interval 10s
--health-timeout 5s
--health-retries 5
redis:
  image: redis:7
  ports: ['6379:6379']
  options: >-
    --health-cmd "redis-cli ping"
    --health-interval 10s
    --health-retries 5
env:
  DATABASE_URL: postgres://postgres:postgres@localhost:5432/app_test
  REDIS_URL: redis://localhost:6379
steps:
  - uses: actions/checkout@v4
  - uses: actions/setup-python@v5
    with: { python-version: '3.11', cache: pip }
  - run: pip install -r requirements.txt
  - run: pytest -q tests/integration
```



YAML

Copy

```
jobs:
  test:
    runs-on: ubuntu-latest
    container:
      image: python:3.11-slim
      options: --user 1001
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: postgres }
    env:
      DATABASE_URL: postgres://postgres:postgres@postgres:5432/postgres
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest -q
```

The hostname changes

With no `container:` , services are reachable at `localhost` . Once the job runs `inside` a container, they are reachable at the **service name**: `postgres:5432` . Moving a job into a container breaks every connection string that said `localhost` , with no warning.

► TRY IT

1. Take a working job with a Postgres service and add `container: { image: python:3.11-slim }` .
2. Run it without changing the connection string.
3. Change `localhost` to `postgres` and run again.

Expected result – the first run fails with a connection error even though nothing else changed. That hostname switch is invisible in a diff and costs people an



.github/workflows/reusable-ci.yml

Copy

```
on:
  workflow_call:
    inputs:
      python-version:
        description: Python version for lint and tests
        type: string
        default: '3.11'
      run-e2e:
        description: Also run the end-to-end suite
        type: boolean
        default: false
    secrets:
      CODECOV_TOKEN:
        required: false
    outputs:
      coverage:
        description: Line coverage percentage
        value: ${{ jobs.test.outputs.coverage }}

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    outputs:
      coverage: ${{ steps.cov.outputs.pct }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
```



```
- id: cov
  run: |
    pytest -q --cov --cov-report=term
    echo "pct=$(coverage report --format=total)" >> "$GITHUB_OUTPUT"
- if: inputs.run-e2e
  run: pytest -q tests/e2e
```

The caller shrinks to almost nothing:

That doubled path, `my-org/.github/.github/workflows/...`, is correct and surprises everyone exactly once: the first `.github` is the **repository** name, the second is the folder inside it.

► TRY IT

1. Create a `reusable-ci.yml` in the same repository with one `input` and one `output`, then call it from another workflow with `uses: ./github/workflows/reusable-ci.yml`.



4. Now pass a secret two ways: explicitly, then with `secrets: inherit`.

Expected result – the local path form works for practice without publishing anything. The permissions experiment fails: the callee cannot exceed the caller's token, which is the constraint that explains most reusable-workflow permission errors.

Composite actions

.github/actions/setup/action.yml

Copy

```
name: Set up project
description: Python, cached dependencies, and our standard tooling
inputs:
  python-version:
    description: Python version to install
    default: '3.11'
  install-dev:
    description: Also install dev requirements
    default: 'false'
outputs:
  cache-hit:
    description: Whether the dependency cache was an exact hit
    value: ${{ steps.py.outputs.cache-hit }}
runs:
  using: composite
  steps:
    - id: py
      uses: actions/setup-python@v5
      with:
        python-version: ${{ inputs.python-version }}
        cache: pip

    - run: pip install -r requirements.txt
```



```
run: pip install -r requirements-dev.txt
shell: bash
```

YAML

Copy

```
steps:
  - uses: actions/checkout@v4
  - uses: ../github/actions/setup          # local path
    with:
      python-version: '3.12'
      install-dev: 'true'
```

✓ Reusable workflow: job level

- Brings its own jobs, runners, `permissions`
- Typed `inputs`, explicit `secrets`, `outputs`
- For a whole pipeline stage
- Receives the caller's token and cannot exceed it

✗ Composite action: step level

- Runs in the caller's job on the caller's runner
- Cannot define jobs, matrices, or its own runner
- For a repeated sequence of steps
- `shell:` required on every `run`

Forgetting `shell:` is the most common reason a new composite action refuses to load, and the error does not say so.

► TRY IT

1. Create `../github/actions/setup/action.yml` as above but **omit** `shell: bash` on one `run` step.
2. Reference it with `- uses: ../github/actions/setup` and run.
3. Add the missing `shell:` and run again.
4. Read one of its `outputs` in the calling workflow.

Expected result – the first run fails to even load the action, with a message that does not mention `shell`. This is the single most common reason a new composite action refuses to work.



YAML

Copy

```
jobs:
  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://app.example.com      # shown on the run and the deployment
    steps:
      - env:
          TOKEN: ${ secrets.DEPLOY_TOKEN }  # environment-scoped secret
        run: ./deploy.sh
```

Configured in **Settings** → **Environments**, an environment gives you:

Feature	Effect
Required reviewers	The job pauses before its first step until a human approves
Wait timer	A forced delay before the job starts
Deployment branches	Only listed branches or tags may deploy
Scoped secrets and variables	Reachable only from a job that declared this environment
Deployment history	A record per deploy, with commit, actor, and URL

The right answer to "keep this credential away from CI"

Put it on the **environment**, not the repository. Ordinary test jobs then cannot read it at all, because only a job that declares `environment: production`, and passed its approval gate, gets it.



1. Create an environment called `production` and add yourself as a required reviewer.
2. Add `environment: production` to a deploy job and trigger the workflow.
3. Watch the job pause, then approve it.
4. Add a secret to the environment and try to read it from a job that does **not** declare the environment.

Expected result – the job waits before its first step and records who approved it. The environment secret is empty in the other job, the mechanism for keeping a production credential away from ordinary CI.

Concurrency

Beginner added `timeout-minutes` as a guard. Concurrency is the other one, and it is usually the single biggest saving available on a busy repository.

YAML

Copy

```
# Workflow level: one run per branch, cancel the superseded one
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: ${{ github.event_name == 'pull_request' }}

jobs:
  deploy:
    runs-on: ubuntu-latest
    concurrency:
      group: deploy-production      # job level: one deploy at a time
      cancel-in-progress: false     # QUEUE rather than kill a release
    steps:
      - run: ./deploy.sh
```

The group key is the whole decision



cancellation you get one deploy at a time, queued, correct for CD. Getting these backwards either kills a release halfway through or serialises every pull request behind every other one.

Because they need different keys, a real pipeline declares both, at different scopes.

► TRY IT

1. Add workflow-level concurrency keyed on `github.ref` with `cancel-in-progress: true`, and a job that sleeps 60 seconds.
2. Push twice in quick succession and watch the first run.
3. Now add a deploy job with its own group and `cancel-in-progress: false`, and push twice again.

Expected result – the first CI run is cancelled the moment the second starts. That is the saving. The deploy job instead **queues**, which is what you want for a release. Seeing both makes the group-key decision concrete.

Path filters and the required-check trap

YAML

Copy

```
on:
  pull_request:
    paths-ignore: ['docs/**', '**/*.md']
  push:
    branches: [main]
    paths: ['src/**', 'requirements.txt']
```

A skipped required check blocks the pull request forever



with the **same job name**, triggered on the excluded paths, that succeeds immediately.

► TRY IT

1. Make your `test` job a required status check in branch protection.
2. Add `paths-ignore: ['**/*.md']` to the `pull_request` trigger.
3. Open a pull request that changes only the README.
4. Add the same-named stub workflow and re-open the pull request.

Expected result – the doc-only pull request sits on "Expected: waiting for status" forever, unmergeable, with no error anywhere. The stub reports immediately and unblocks it. Worth causing once on purpose.

Triggers in depth

Beyond the basics, three triggers unlock patterns you cannot build otherwise.

YAML

Copy



```
WORKFLOW_RUN.  
  workflows: ['CI']  
  types: [completed]  
  branches: [main]  
  
jobs:  
  publish:  
    if: github.event.workflow_run.conclusion == 'success'  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/download-artifact@v4  
        with:  
          name: dist  
          run-id: ${ github.event.workflow_run.id }  
          github-token: ${ secrets.GITHUB_TOKEN }
```

YAML

Copy

```
# Triggered from outside GitHub, by an API call  
on:  
  repository_dispatch:  
    types: [deploy-request]  
  
jobs:  
  deploy:  
    runs-on: ubuntu-latest  
    steps:  
      - run: ./deploy.sh --env "${ github.event.client_payload.environment }"
```

BASH

Copy

```
curl -X POST \  
  -H "Authorization: Bearer $TOKEN" \  
  -H "Accept: application/vnd.github+json" \  
  https://api.github.com/repos/OWNER/REPO/dispatches \  
  -d '{"event_type":"deploy-request","client_payload":{"environment":"staging"}}'
```



<code>pull_request</code> types	Default is <code>opened</code> , <code>synchronize</code> , <code>reopened</code> . Add <code>ready_for_review</code> , <code>labeled</code> as needed
<code>schedule</code>	UTC only, default branch only, and queued rather than <code>punctual</code> on busy repositories
<code>workflow_run</code>	Runs in the base repository context with a writable token: see Senior
<code>repository_dispatch</code> h	Needs a token with <code>contents: write</code> ; payload arrives as <code>client_payload</code>

► TRY IT

1. Build a two-workflow chain: the first uploads an artifact, the second uses `workflow_run` and downloads it with `run-id` .
2. Gate the second on `github.event.workflow_run.conclusion == 'success'` , then make the first one fail.
3. Add a `repository_dispatch` trigger and fire it with the `curl` command above.

Expected result – the chained workflow runs only after success, and the dispatch fires from outside GitHub entirely. Note the second workflow runs against the **default branch** version of itself, the detail that matters for security at Senior level.

Debugging, one level deeper

1 Read the trigger first

Half of "it did not run" is a branch filter, a path filter, or a fork pull request.

2 Dump the context

`run: echo '${{ toJSON(github) }}'` answers most "why didn't my condition match" questions in one run.



4 Compare against the last green run

The difference is often environmental: runner image version, an action release, a cache hit that became a miss.

5 Bypass the cache

Delete the entry from the Caches page or change the key. Stale caches produce failures that make no sense against the code.

6 Bisect in a scratch workflow

Copy the failing job into its own file with `workflow_dispatch` and delete half the steps at a time.

YAML

Copy

```
# Worth keeping permanently: diagnostics that only run after a failure
- name: Diagnostics
  if: failure()
  run: |
    echo "runner image : $ImageVersion"
    echo "ref / sha      : $GITHUB_REF / $GITHUB_SHA"
    echo "cache hit      : ${ steps.deps.outputs.cache-hit }"
    df -h && free -m
```

► TRY IT

1. Add a step that dumps `toJSON(github.event)` and run it on both a push and a pull request.
2. Re-run one job with **Enable debug logging** ticked and compare the log length.
3. Add the `if: failure()` diagnostics step and force a failure.
4. Delete a cache entry from **Actions** → **Caches** and re-run.

Expected result – the event payloads are strikingly different between the two triggers, which explains most condition bugs. The diagnostics step gives you runner image, ref, and cache state for free on every future failure.



Every topic from this level in one pipeline.

.github/workflows/ci.yml

Copy

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    paths-ignore: ['docs/**', '**/*.md']
  workflow_dispatch:
    inputs:
      run-e2e:
        type: boolean
        default: false

concurrency:
  group: ${ github.workflow }-${ github.ref }
  cancel-in-progress: ${ github.event_name == 'pull_request' }

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    strategy:
      fail-fast: false
    matrix:
      python: ['3.11', '3.12']
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: postgres }
        ports: ['5432:5432']
        options: >-
          --health-cmd pg_isready --health-interval 10s --health-retries 5
    env:
      DATABASE_URL: postgres://postgres:postgres@localhost:5432/postgres
    outputs:
```




```
- uses: ./github/actions/setup
  with:
    python-version: ${ matrix.python }
- id: cov
  run: |
    pytest -q --cov --cov-report=xml --junitxml=reports/junit-${ matrix.python }
    echo "pct=$(coverage report --format=total)" >> "$GITHUB_OUTPUT"
- if: always()
  uses: actions/upload-artifact@v4
  with:
    name: reports-${ matrix.python }
    path: reports/
    retention-days: 7
    if-no-files-found: error
- if: always()
  run: echo "| py${ matrix.python } | ${ steps.cov.outputs.pct }%|" >> "$GITH
```

build:

```
needs: test
if: github.ref == 'refs/heads/main'
runs-on: ubuntu-latest
outputs:
  version: ${ steps.meta.outputs.version }
steps:
- uses: actions/checkout@v4
- id: meta
  run: echo "version=1.4.${ github.run_number }" >> "$GITHUB_OUTPUT"
- run: make build
- uses: actions/upload-artifact@v4
  with: { name: dist, path: dist/ }
```

deploy:

```
needs: build
runs-on: ubuntu-latest
environment:
  name: production
  url: https://app.example.com
concurrency:
  group: deploy-production
  cancel-in-progress: false
steps:
- uses: actions/download-artifact@v4
  with: { name: dist, path: dist/ }
- env:
```



```
report:
  needs: [test, build, deploy]
  if: always()
  runs-on: ubuntu-latest
  steps:
    - run: |
        echo "test    : ${ needs.test.result }"
        echo "build   : ${ needs.build.result }"
        echo "deploy  : ${ needs.deploy.result }"
```

Senior takes the same topics one level further and adds what you own when CI/CD is your **responsibility**. Every topic above reappears with a security or scale dimension:



Triggers	The trust model , which triggers give fork code a privileged token
Secrets	OIDC, so there is no stored credential to leak
<code>permissions</code> (mentioned only)	Least privilege in full, and why declaring one scope zeroes the rest
Actions you consume	Actions you author : composite, JavaScript, Docker
Reusable workflows	Running them as a versioned platform across sixty repositories
Runners	Self-hosted strategy, ephemerality, and the cost model
Caching	Container layer caching with Buildx, and cache poisoning
Debugging	Observability metrics, and incident playbooks
Expressions	Script injection, and why event data never touches a shell



PREVIOUS

Beginner · Tips & Tricks

NEXT UP

Mid-level · Interview Review





LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

